Advanced Systems Lab - Team #26

Anastasija Tortevska, Glenn Schönbächler,
Kristijan Zafirovski, Mukhammadali Sayfiddinov

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Agenda

- EzGatr – Easy Geometric Algebra Transformer
- Optimization Outline
- Function Optimization Examples
- Results
- Q&A

EzGATr

- Specialized **Equivariant** Transformer over **Projective Geometric Algebra** with 16-component multivectors

$$\text{EquiLinear}(x) = \sum_k w_k \langle x \rangle_k + \sum_k v_k \, e_0 \langle x \rangle_k$$

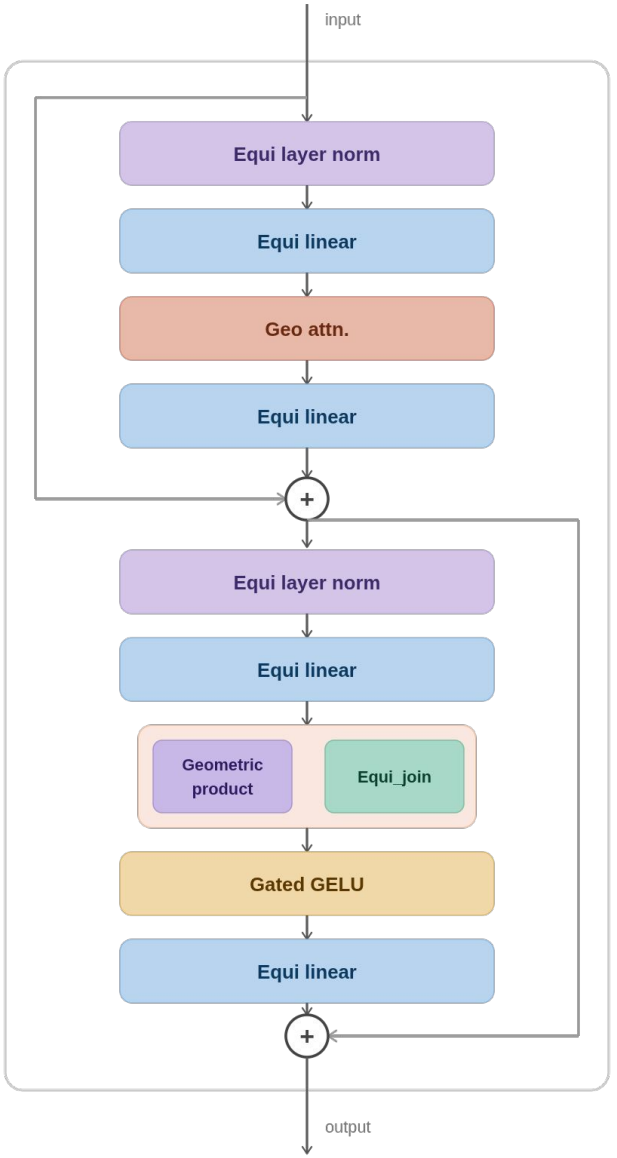
$$\text{EquiJoin}(x, y; z) = z_{0123} \, (x^* \wedge y^*)^*$$

$$\text{GeoProduct}(x, y) = \langle x, y \rangle + x \wedge y$$

$$\text{GatedGELU}(x) = \text{GELU}(x_1) \, x$$

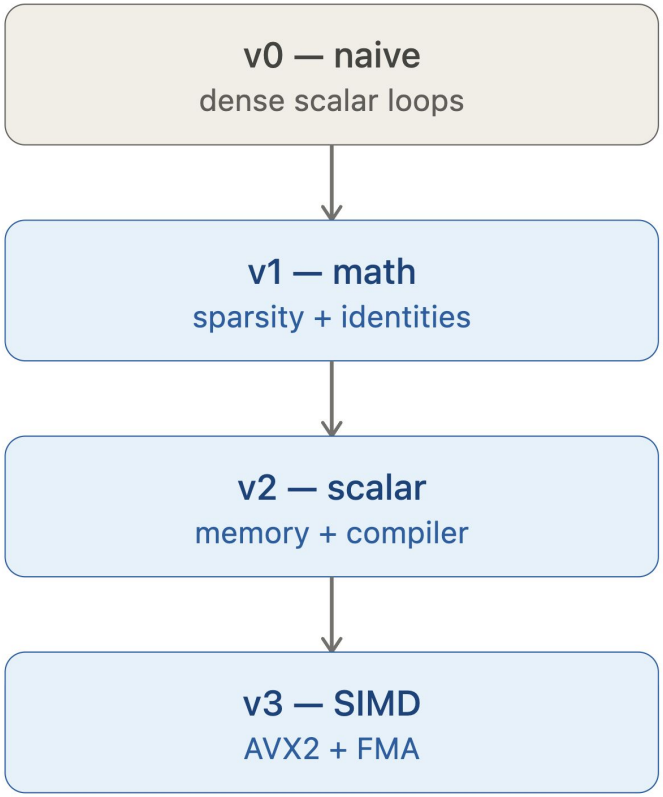
$$\text{LayerNorm}(x) = \frac{x}{\sqrt{\mathbb{E}_c \langle x, x \rangle}}$$

$$\text{Attn}(q, k, v)_{i'} = \sum_i \text{softmax}_i \left(\frac{\sum_c \langle q_{i'c}, k_{ic} \rangle}{\sqrt{8 n_c}} \right) v_i$$



General Optimization Outline

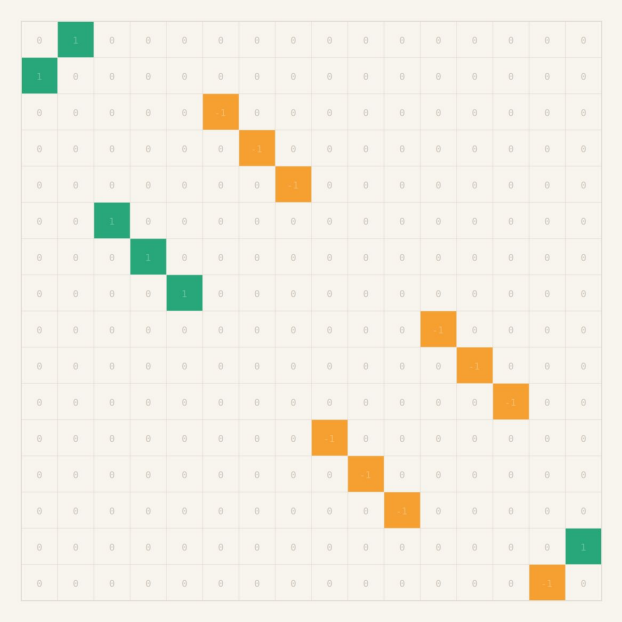
- **Goal**: replace selected EzGATr components with CPU-optimized C/C++ kernels
- Exploit **sparsity**, **sign patterns**, and **fixed structure** for faster inference
- Same optimization framework applied to every EzGATr function
- Assigned functions to team members
- **Tested** each optimized version and compared and **verified** against the PyTorch output



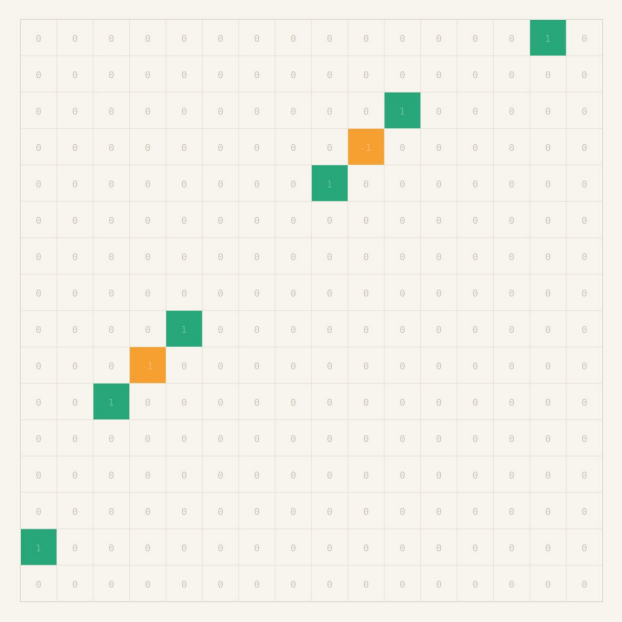
Function Optimization Examples

Function analysis: GeometricBilinears

- **The problem with geometric_product and equi_join**
  - Dense triple loop over the (16,16,16) blades:
  - 4096 FMAs per multivector
- **The room:**



- GP blade, **192/4096 (~5%)** non-zeros overall



- Join blade, **81/4096 (~2%)** non-zeros overall

geometric_product and equi_join v1 - v2

- **v1 - build a list of non-zero entries {i,j,k, sign}** [14x, 34x]
  - loop over list and update output mv
  - for all m:

    o[e[m].i] += e[m].sign * x[e[m].j] * y[e[m].k]

- **v2 - unroll kernels per-blade** [500x, 680x]
  - 16 straight line evaluations and stores
  - o[4] = x[0]*y[3] + x[2]*y[8] + … + x[10]*y[4] + x[14]*y[9]
  - __restrict__ to avoid aliasing

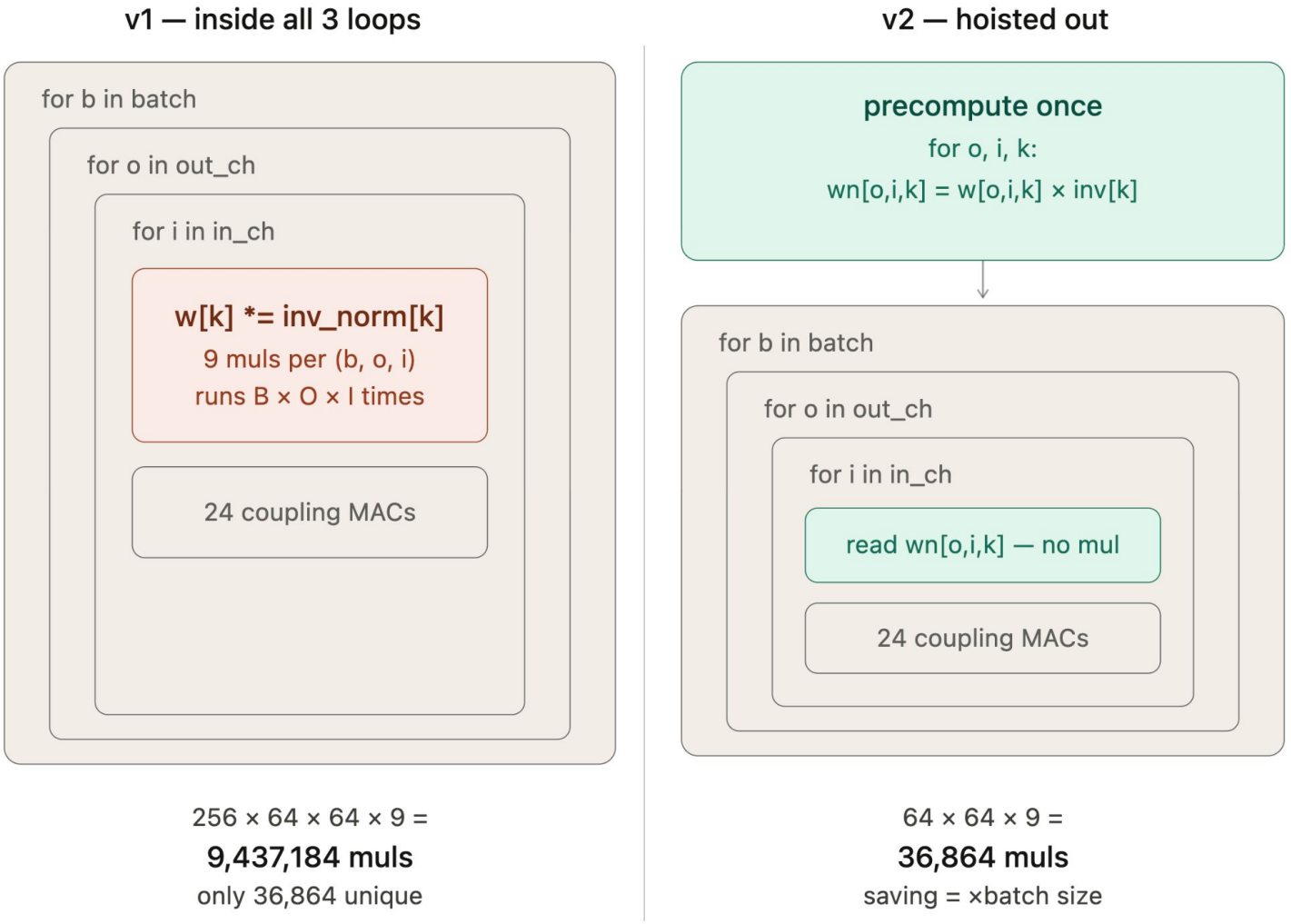
geometric_product and equi_join v2.x - v3

- **v2.x - loop unroll & accumulate, SSA per blade, tiling**
  - modest gains ~ 10% for join with SSA
- **v3 - SIMD**
  - almost **random, non-contiguous access** hampers vectorization
  - was implemented with **transpose** but didn't justify the cost

equi_linear v2 — Remove Redundant Work

- **Hoist Normalization**
  - w[k] × inv_norm[k] depends only on (o, i), never on batch item b
  - v1 recomputes it inside all 3 loops: B × O × I × 9 multiplications
  - v2 precomputes once before the batch loop: O × I × 9 multiplications
  - Saving scales exactly with batch size

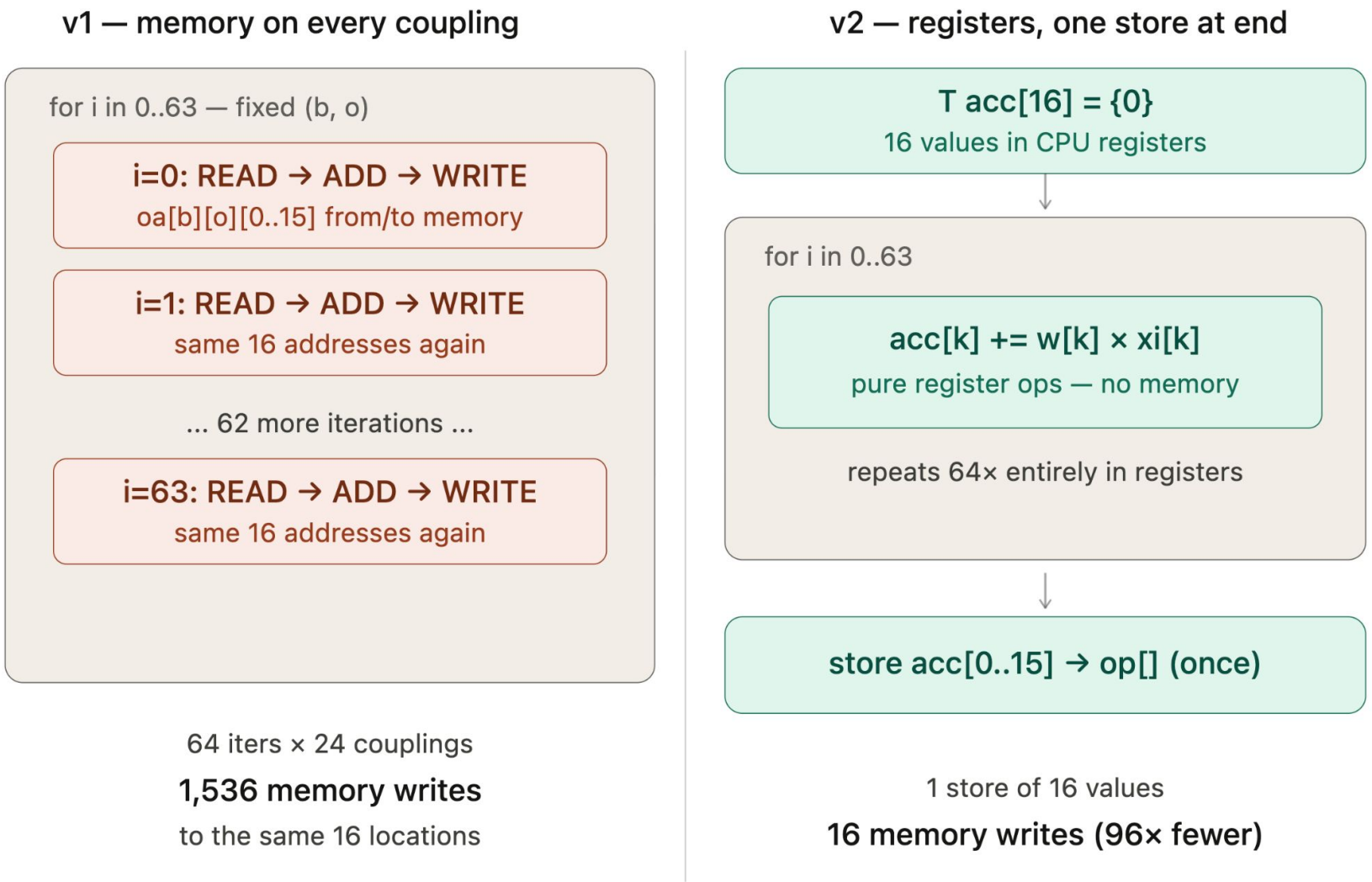
equil_linear v2 — Remove Redundant Work



equil_linear v2 — Reduce Memory Traffic

- **Register-Resident Accumulator**
  - v1: every += is a read-modify-write to memory
  - 64 iters × 24 couplings = 1,536 writes to the same 16 addresses per (b, o)
  - v2: T acc[16] lives in CPU registers, then accumulate across all input channels, store once. 16 writes total
  - 96× fewer memory writes per (b, o) pair

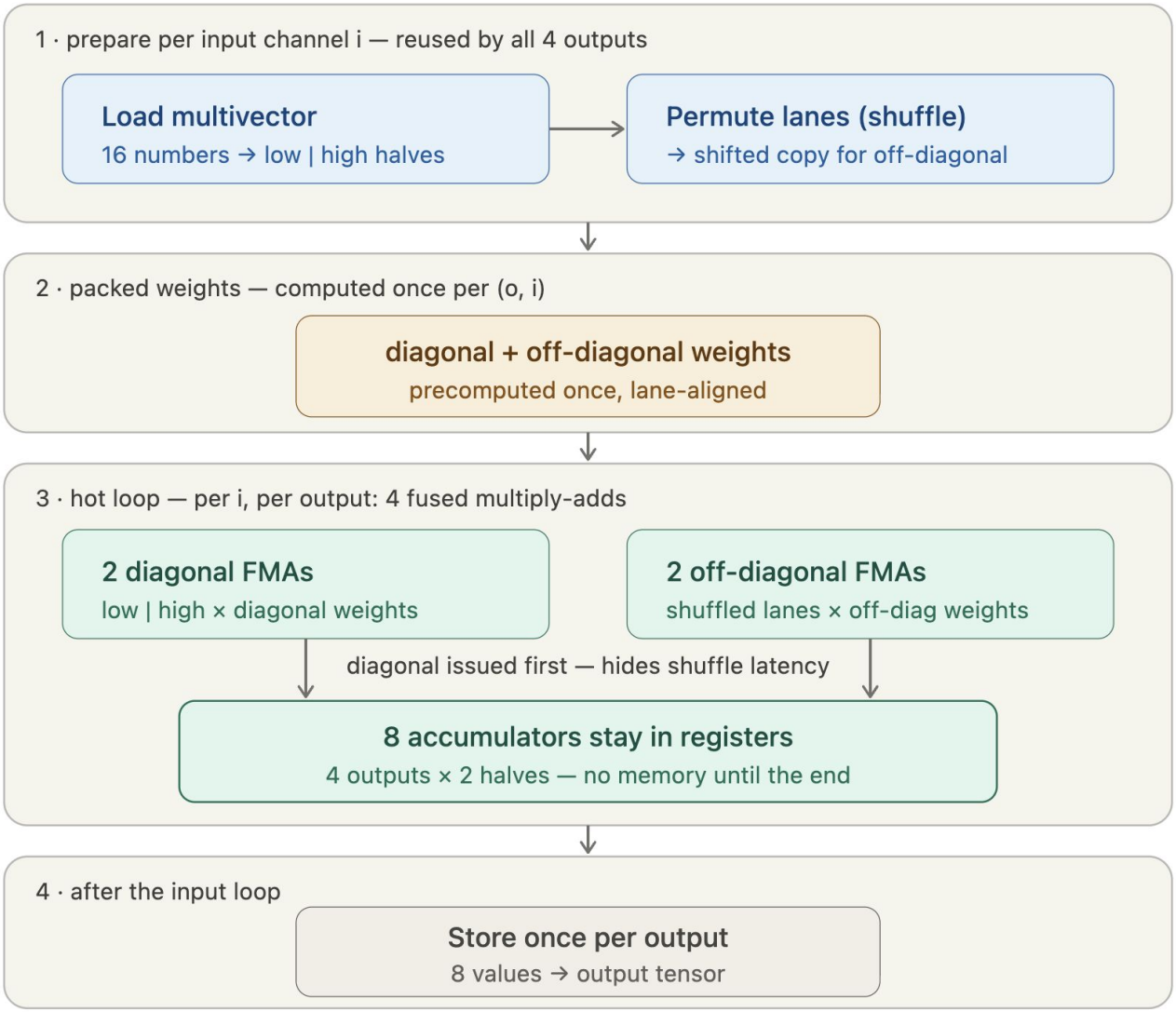
equil_linear v2 — Reduce Memory Traffic



equi_linear v3 — AVX2 vectorization

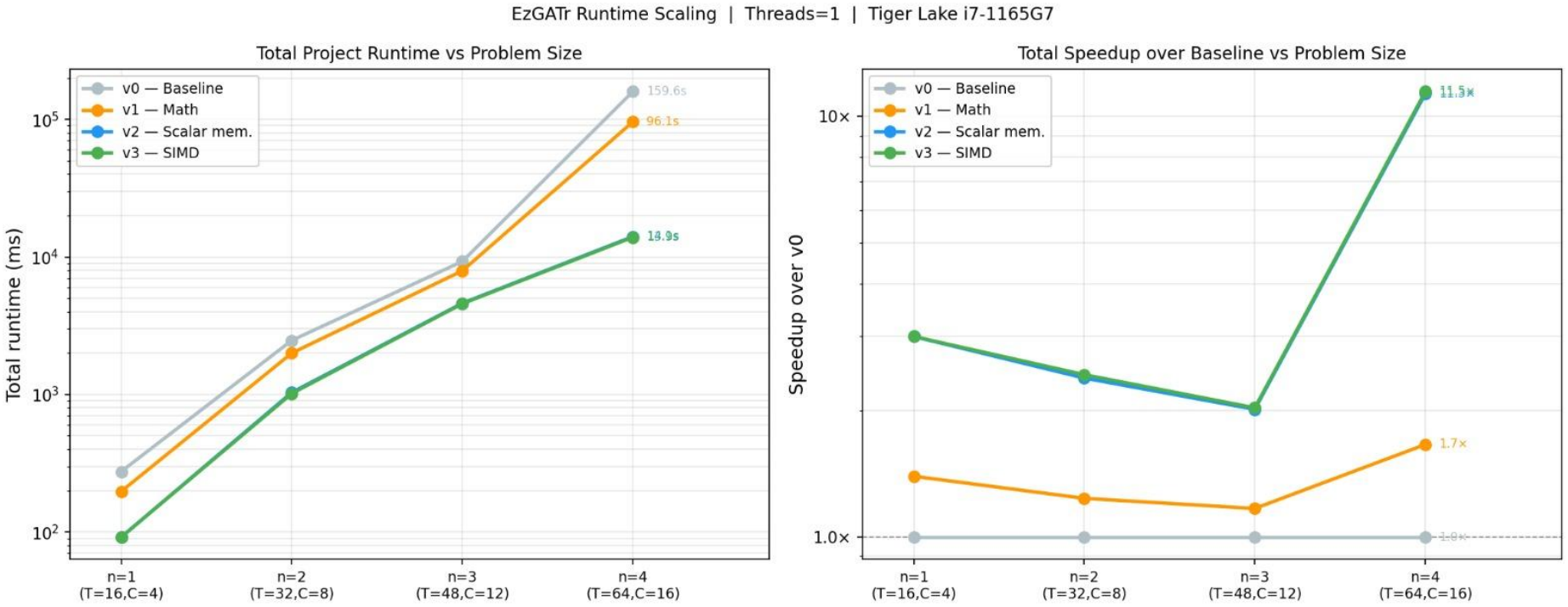
- **SIMD data layout**
  - 16-number multivector → two 8-lane registers weights pre-packed per (output, input) into lane-aligned vectors.
- **Diagonal vs Shifted couplings**
  - 16 of 24 couplings are direct lane-wise multiplies, the other 8 use two in-register shuffle instead of scattered memory reads.
- **FMA**
  - Each coupling is acc = acc + weight × input in one fused instruction
    → 24 scalar mul-adds become 4 vector FMAs per channel pair.
- **Scheduling & Reuse**
  - Load + shuffles done once per (batch, input), reused across 4 outputs, 8 accumulators stay in registers, one store at the end.
- **Net effect**
  - ~6× fewer arithmetic instructions per (output, input) pair
    24 mul-adds → 4 FMAs

v3 — AVX2 vectorization

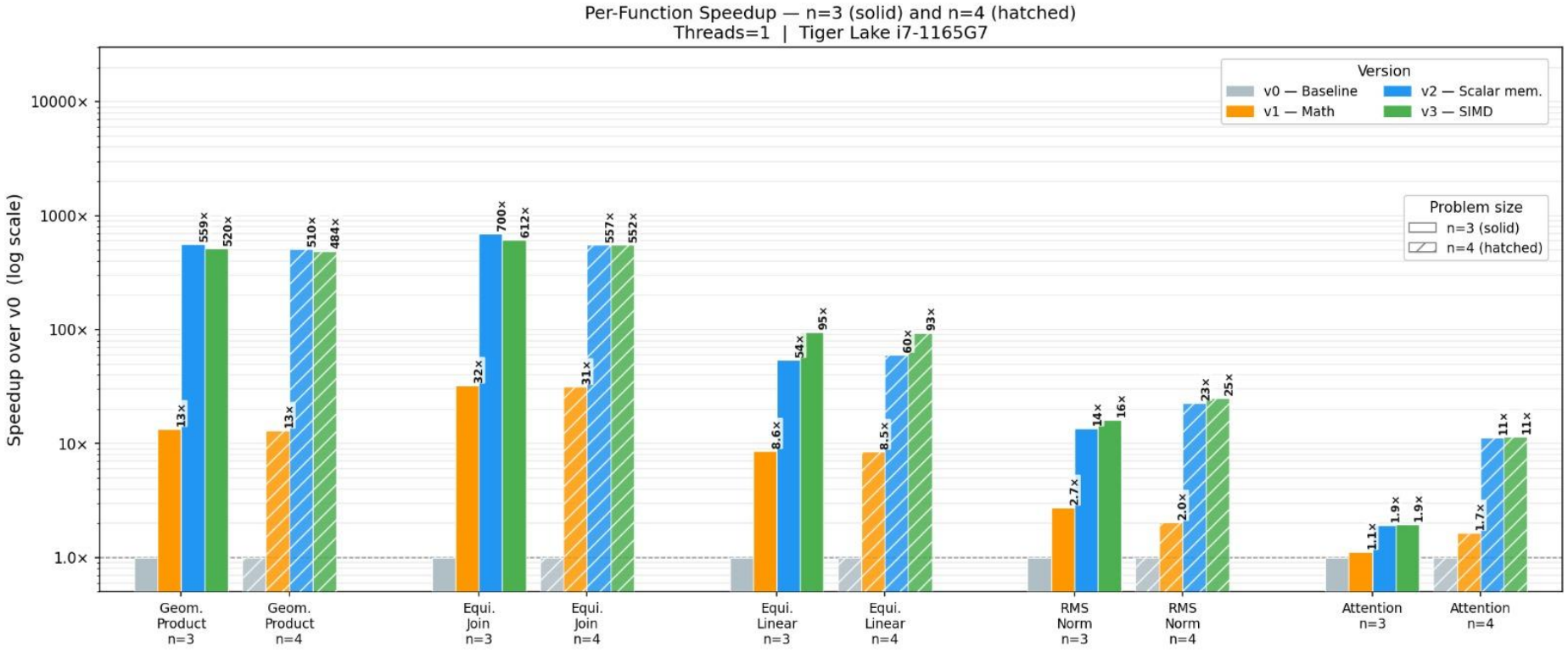


Results

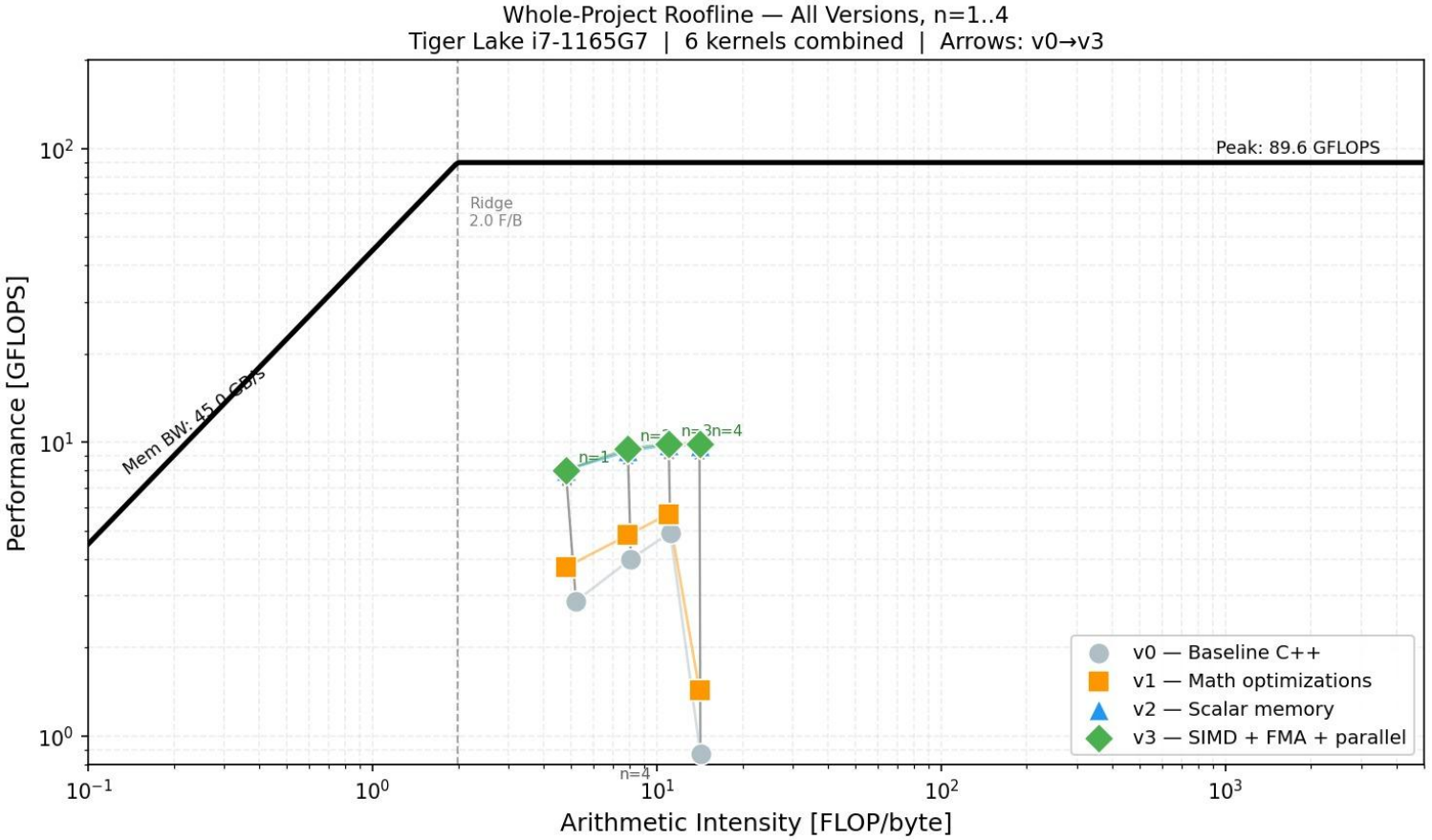
Results – End-to-End Runtime Scaling



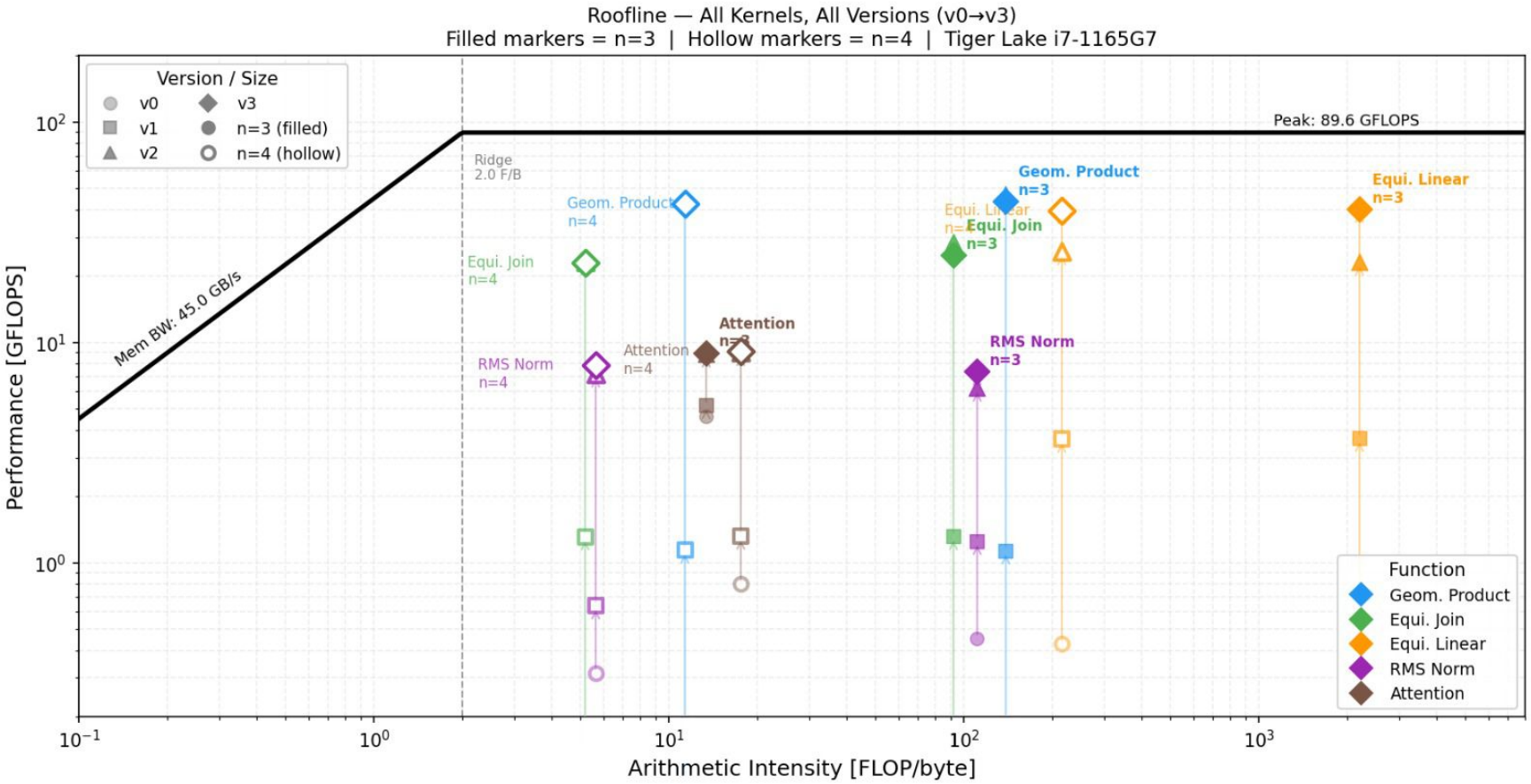
Results – Per-Function Speedup



Results – End-to-End Roofline Plots



Results – Roofline Analysis



Thanks for all the

Attention

Q&A

Results — Best Per-Function Speedups (n=1)

Function	Baseline	Best version	Speedup
geometric_product	19.799 ms	v2 = 0.034 ms	582.3×
equi_join	19.907 ms	v2 = 0.030 ms	663.6×
equi_linear	2.211 ms	v3 = 0.031 ms	71.3×
equi_rms_norm	0.223 ms	v3 = 0.023 ms	9.7×
equi_geometric_attention	186.267 ms	v3 = 64.297 ms	2.9×

For n=1, the largest improvements come from the sparse geometric algebra kernels, with equi_join and geometric_product reaching over 580× speedup.

Results — Best Per-Function Speedups (n=2)

Function	Baseline	Best version	Speedup
geometric_product	80.495 ms	v2 = 0.139 ms	579.5×
equi_join	80.511 ms	v3 = 0.130 ms	617.1×
equi_linear	21.279 ms	v3 = 0.217 ms	98.0×
equi_rms_norm	1.169 ms	v3 = 0.208 ms	5.6×
equi_geometric_attention	2346.666 ms	v3 = 1072.443 ms	2.2×

At the n=2 problem size, the same trend remains: geometric_product and equi_join still exceed 570x speedup, while equi_linear improves further to 98x.

Results — Best Per-Function Speedups (n=3)

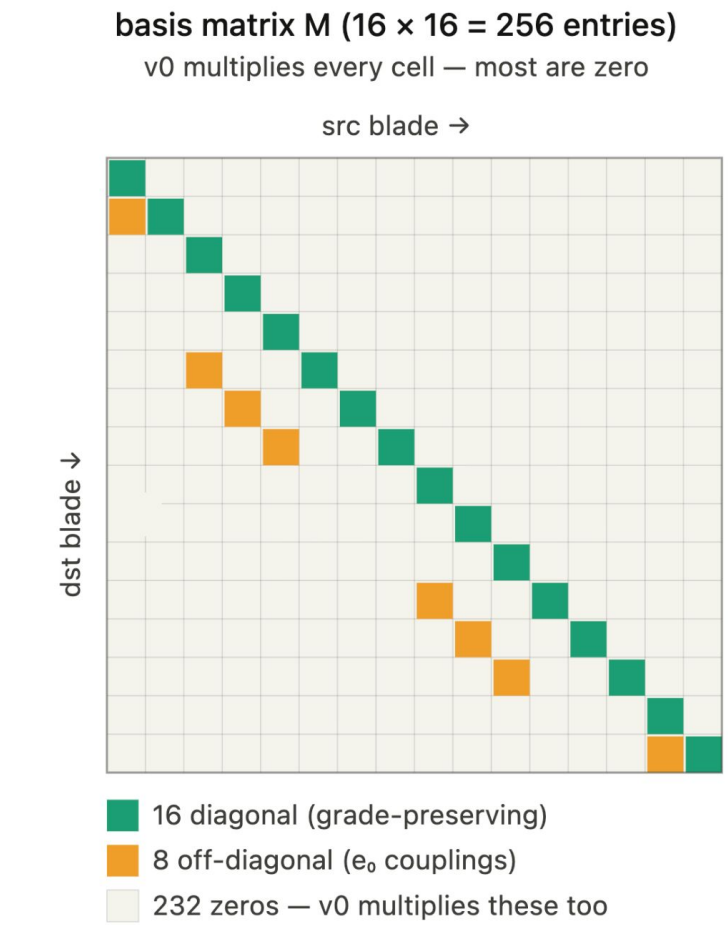
Function	v0	v1	v2	v3	v0/v1	v0/v2	v0/v3
geometric_product	182 ms	13 ms	0.36 ms	0.34 ms	13.9×	504×	530×
equi_join	184 ms	5.5 ms	0.27 ms	0.30 ms	33.5×	686×	604×
equi_linear	65.9 ms	7.9 ms	1.27 ms	0.71 ms	8.3×	52×	92×
equi_rms_norm	3.28 ms	0.88 ms	0.18 ms	0.16 ms	3.7×	18×	21×
equi_geometric_attention	9592 ms	9563 ms	4901 ms	4891 ms	1.0×	2.0×	2.0×

Most kernel-level speedups come from combining geometric algebra simplification in v1 with CPU memory/layout and SIMD improvements in v2/v3.

equi_linear v1: Exploiting Sparsity

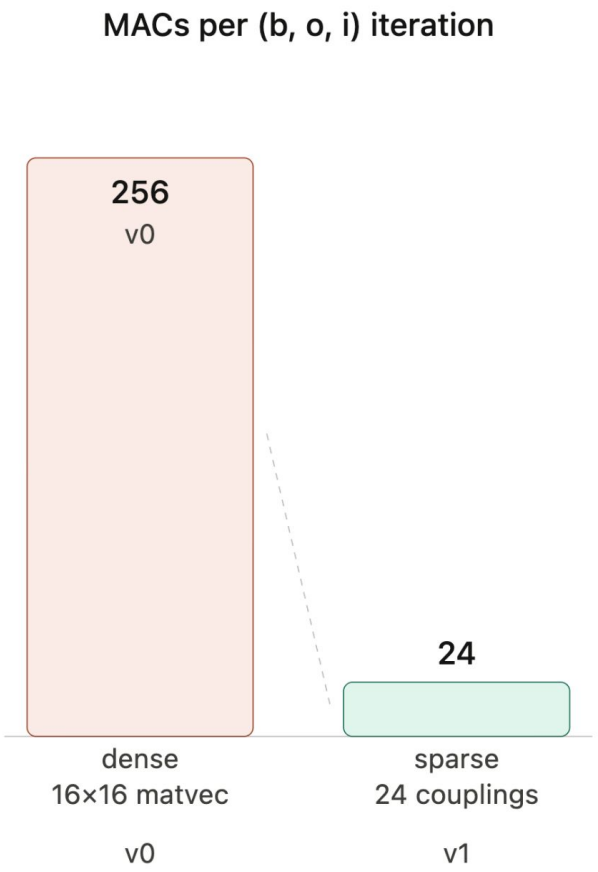
- **The problem**
  - Python einsum builds a full 16×16 weight matrix M per (o, i) pair, then runs a dense matrix-vector multiply
  - 256 multiply-accumulates per (b, o, i) iteration, most multiplying by zero
- **The fix**
  - The equivariant basis has only 24 nonzero couplings out of 256 possible
  - Skip the matrix entirely, hand-unroll just the 24 nonzero terms directly
  - 16 diagonal (grade-preserving) + 8 off-diagonal (e_0-related)

equil_linear v1: Exploiting Sparsity



v0: out[d] = Σ_s M[d][s] · x[s] (256 terms, most = 0)

v1: out[dst] += w[k] · x[src] for each of 24 nonzero couplings



~10× fewer FLOPs

same mathematical result

232 zero multiplications eliminated